SharpaWave with Manus

User Manual v1.0

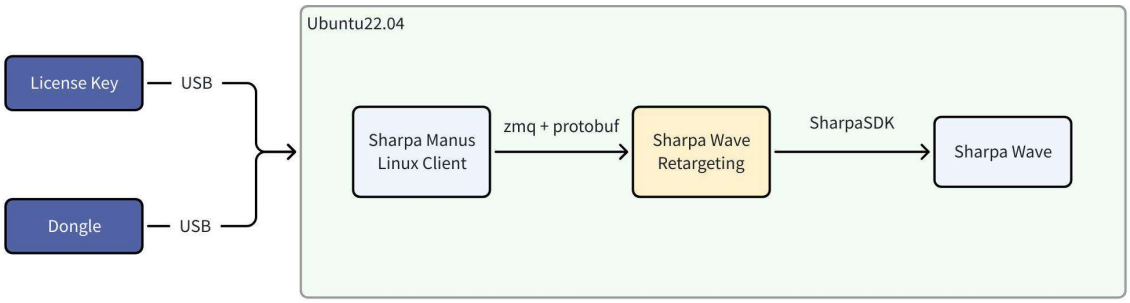
sharpawave

Contents

1 Overiew....4
1.1 Hardware requirements....4
1.2 Software requirements....4
2 Setup Guide....6
2.1 Hardware setup....6
2.2 Software Setup....6
2.3 Overall Workflow....10
2.4 Calibration for Manus Glove....10
2.5 Installation Notes....13
3 Developer Guide....16
3.1 Overview....16
3.2 RawSkeleton....16
3.3 Data type for retargeting algs....16
3.4 Core functions....21
3.5 Demo....22

1 Overiew

1 Overiew



This system implements a real-time hand motion capture and robotic control system using Manus data gloves. Sharpa Manus Linux Client acquires the key-point data of the human hand from the gloves and publishes it via ZMQ; the retargeting algorithm then generates per-joint angle commands for Sharpa Wave, which are then executed via the Sharpa SDK.

1.1 Hardware requirements

- **Manus MetaGloves Pro (includes SDK License Key and wireless dongle)**
- **Recommended PC configuration:**

Memory	32G
Processor	Intel Core i7-14650HX
Graphics	N/A
Operating System	Ubuntu 20.04 / Ubuntu 22.04 / Ubuntu 24.04, 64-bit

1.2 Software requirements

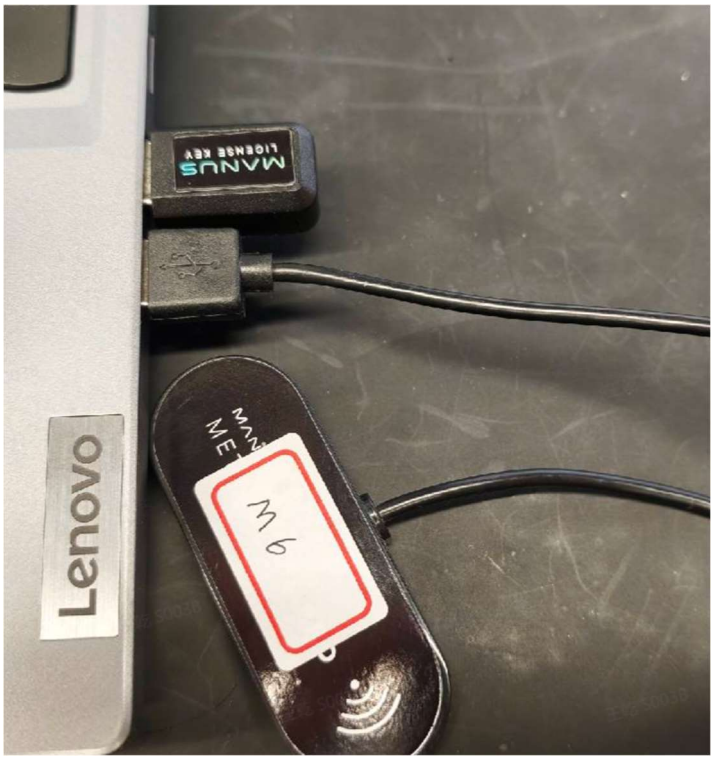
- **Sharpa Manus Client: Manus-glove client for the Sharpa Wave retargeting algorithm; receives and forwards glove data.**
- **Sharpa Wave Retargeting: Remapping algorithm that takes the five-digit pose stream from the Manus gloves, transforms it into the Sharpa Wave joint space, and outputs the corresponding joint angles, which are then executed via the Sharpa SDK.**
- **Sharpa Pilot: Graphical interface for monitoring Sharpa Wave and for switching its operating mode.**

2 Setup Guide

2 Setup Guide

2.1 Hardware setup

- **Power up Sharpa Wave and connect it to the PC via Ethernet cable.**
- **Plug the Manus glove wireless dongle and License Key into the PC.**



- **Switch on the gloves and verify the connection status (LED solid blue).**

2.2 Software Setup

2.2.1 Sharpa Manus Client

> **Our Sharpa Manus Linux Client is based on the official SDK's SDKClient_Linux. We have streamlined the implementation, added coordinate system conversion, and enabled data transmission via ZMQ and Protobuf.**

1. **Configure the PC's wired Ethernet IPv4 address: same as the unicast destination address for tactile data (default: 192.168.10.240), ensuring it does not conflict with the Sharpa Wave IP addresses (default: left hand 192.168.10.10, right hand 192.168.10.20). Refer to**

Sharpa Pilot User Manual, Part 3, for Sharpa Wave IP configuration instructions.

2. **Install protobuf (libprotobuf.so.23) & protoc (3.12.4) & ZMQ**

```
sudo apt-get install -y \
    libprotobuf-dev \
    protobuf-compiler \
    libprotoc-dev \
    libzmq3-dev
```

3. **Install the dependencies required for the Manus SDK**

```
sudo apt-get update && \
sudo apt-get install -y \
    build-essential \
    libusb-1.0-0-dev \
    zlib1g-dev \
    libudev-dev \
    gdb \
    libncurses5-dev \
    libeigen3-dev \
    libglfw3-dev \
    libgl1-mesa-dev \
    libglu1-mesa-dev \
&& sudo apt-get clean
```

Due to the default cppzmq version shipped with Ubuntu 20.04, additional manual steps are required; please refer to Installation Note 2 for details.

4. **To allow connections to MANUS hardware you need to create a file named `70-manus-hid.rules` in the `etc/udev/rules.d/` directory with following commands:**

```
sudo tee /etc/udev/rules.d/70-manus-hid.rules > /dev/null <<'EOF'
# HIDAPI/libusb
SUBSYSTEMS=="usb", ATTRS{idVendor}=="3325", MODE:="0666"
SUBSYSTEMS=="usb", ATTRS{idVendor}=="1915", ATTRS{idProduct}=="83fd",
MODE:="0666"

# HIDAPI/hidraw
KERNEL=="hidraw*", ATTRS{idVendor}=="3325", MODE:="0666"
EOF
```

This will allow the devices to be automatically recognized and accessed by the system. After doing this, a full reboot is recommended to apply the changes.

5. **Extract the SharpaManusLinuxClient.tar.xz archive.**
6. **Compile and run the Sharpa Manus Linux Client.**

```
cd SharpaManusLinuxClient
make
```

7. **Run Sharpa Manus Cient**

```
./SharpaManusClient.out
```

8. **If the terminal continuously prints the following message, it indicates that the Sharpa Manus Linux Client is running normally:**

> INFO: Frame[xx] skeleton: Right is published. - System time: xx.xxs
>
> INFO: Frame[xx] skeleton: Left is published. - System time: xx.xxs

> **Note: if a lower-version protoc and protobuf are already present in the local environment, compilation may fail. If there is any error caused by protobuf during the compilation, please refer to Installation Note 3 for the resolution.**

2.2.2 SharpaWave Retarget

1. **Unzip retargeting_alg.zip and open the extracted folder.**
2. **Creating a Conda Environment**

Create and activate the conda environment using the provided configuration file:

```
conda env create -n sharpa -f environment.yml
conda activate sharpa
```

3. **Start the Retargeting Service**

- **Launch the retargeting service to publish hand actions via ZMQ:**

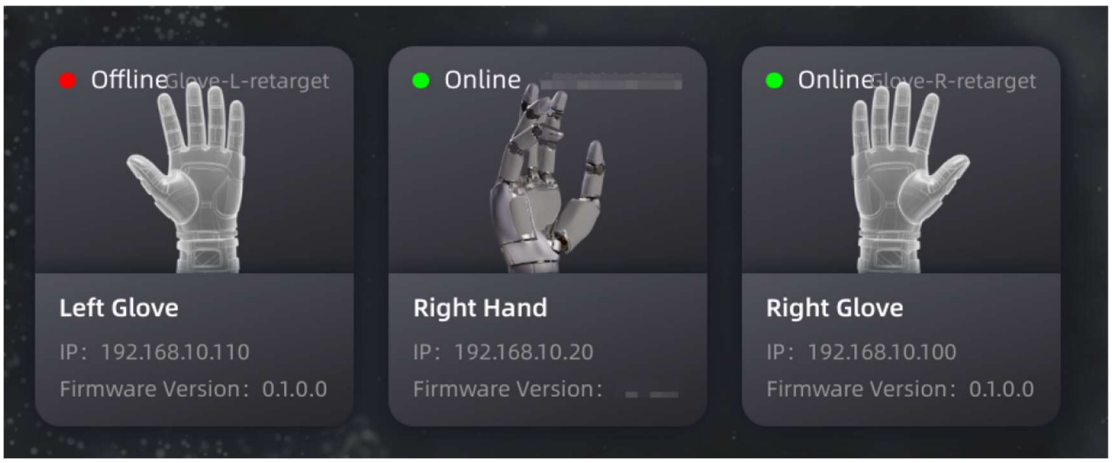
```
python3 retargeting_manus_demo_multiprocess.py
```

- **Launch the retargeting service for use with physical Wave hand hardware:**

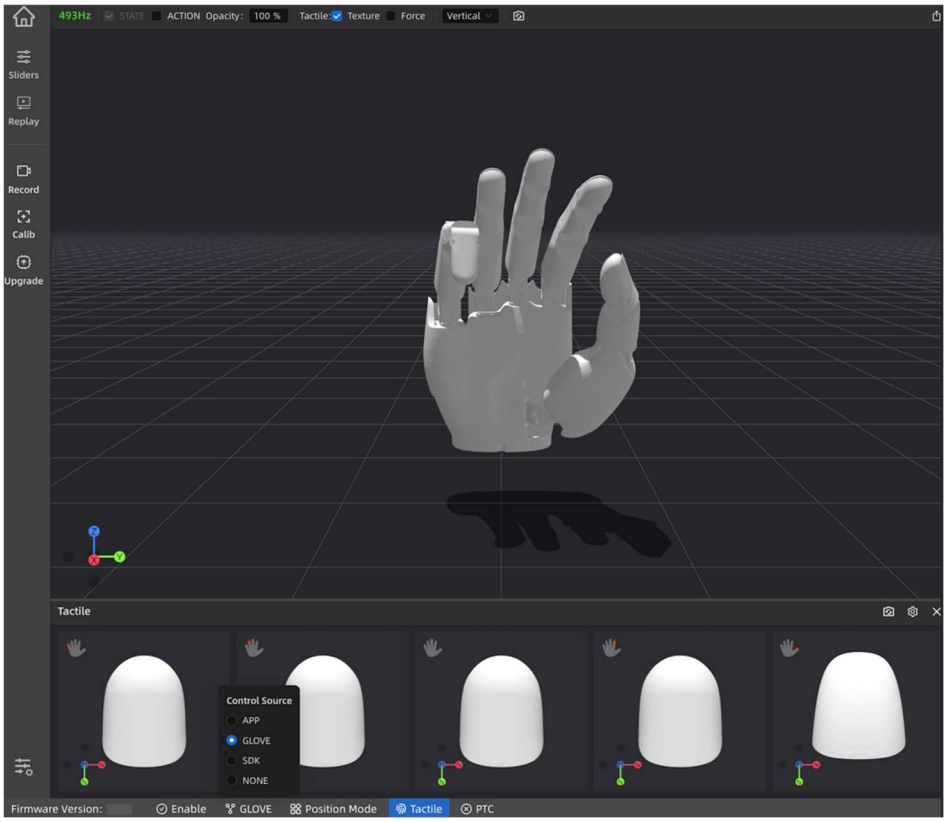
```
python3 retargeting_manus_demo_multiprocess.py -wave -glove_left_ip
192.168.10.98 -glove_right_ip 192.168.10.99
```

The glove IP address must reside in the same subnet as the Sharpa Wave IP; for example, if Sharpa Wave is 192.168.10.10, assign the glove to 192.168.10.11.

4. **Launch Sharpa Pilot, the corresponding Glove and Wave devices will appear. The figure below shows the case where only the right-hand glove is connected, while the left-hand glove is indicated in red as offline.**



5. **Use the control-source switch at the bottom of Pilot to select "Glove" and teleoperation can now begin.**



2.3 Overall Workflow

1. **Setup hardware as introduced in section 2.1.**
2. **Open SharpaManusClient folder and run the client :**

```
./SharpaManusClient.out
```

3. **Open retargeting algorithm folder and launch the retargeting service:**

```
conda activate sharpa
python3 retargeting_manus_demo_multiprocess.py -wave -glove_left_ip
192.168.10.98 -glove_right_ip 192.168.10.99
```

4. **Use Sharpa Pilot to switch the control mode of Sharpa Wave to glove mode.**

2.4 Calibration for Manus Glove

Manus-glove performance varies among individuals, per-user calibration is therefore required. Please refer to the 2.3 Calibration for Manus Glove for the detailed calibration protocol. To ensure optimal glove performance, a per-user hand calibration is mandatory. Note that each "operator-glove" pair requires its own calibration file: if users A and B share glove Alpha, they each need distinct calibration files, likewise, when user A switches from glove Alpha to glove Beta, a new calibration file is required.

For convenience, we provide an Ubuntu calibration program called CalibrationGUI that follows the identical calibration procedure described in Calibration - Knowledge Center.

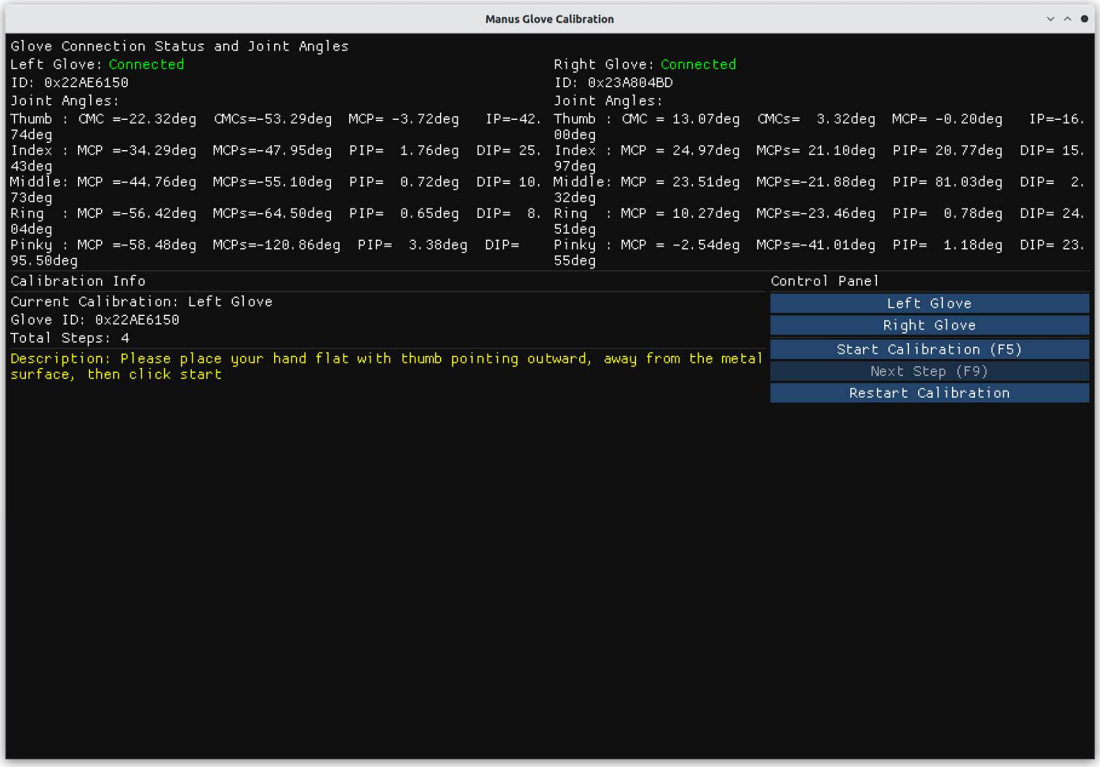
1. **Compile calibrationGUI:**

```
cd SharpaManusLinuxClient/CalibrationGUI
make
```

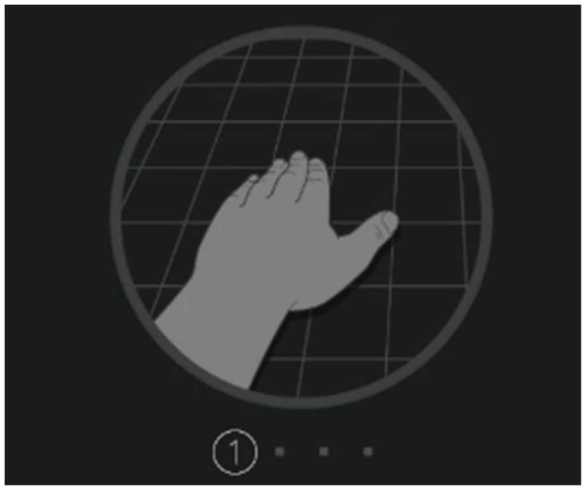
2. **Connect your Manus MetaGloves Pro (with License Key and Dongle)**
3. **Run CalibrationGUI with**

```
./CalibrationGUI.out
```

4. **You should find the states of the gloves on the top**



5. **Place your hand flat as instructed in the description.**



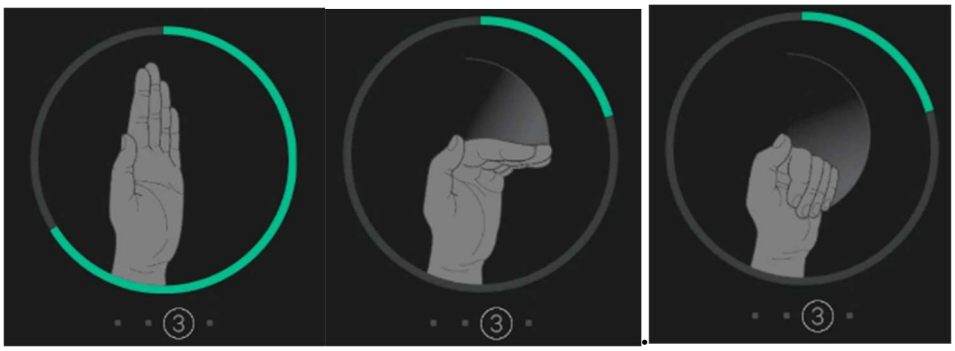
6. **Press F5 to start calibration and wait for 5 seconds.**

7. **Make a fist**

8. **Press F9 and wait for 5seconds**

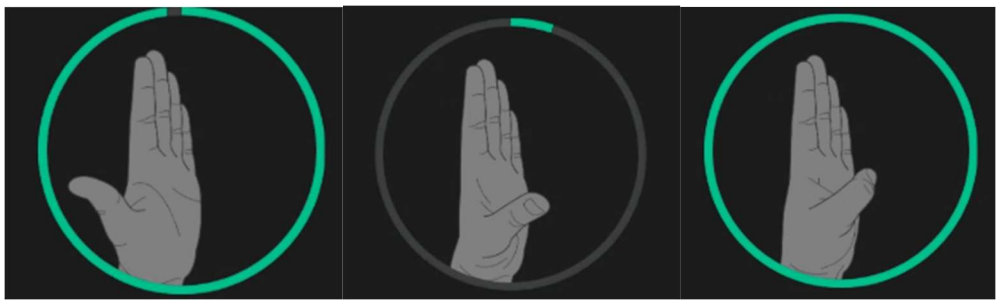
9. **Press F9 for next step**

10. **With your fingertips touch the base of the palm repeatedly until the description give further instruction.**

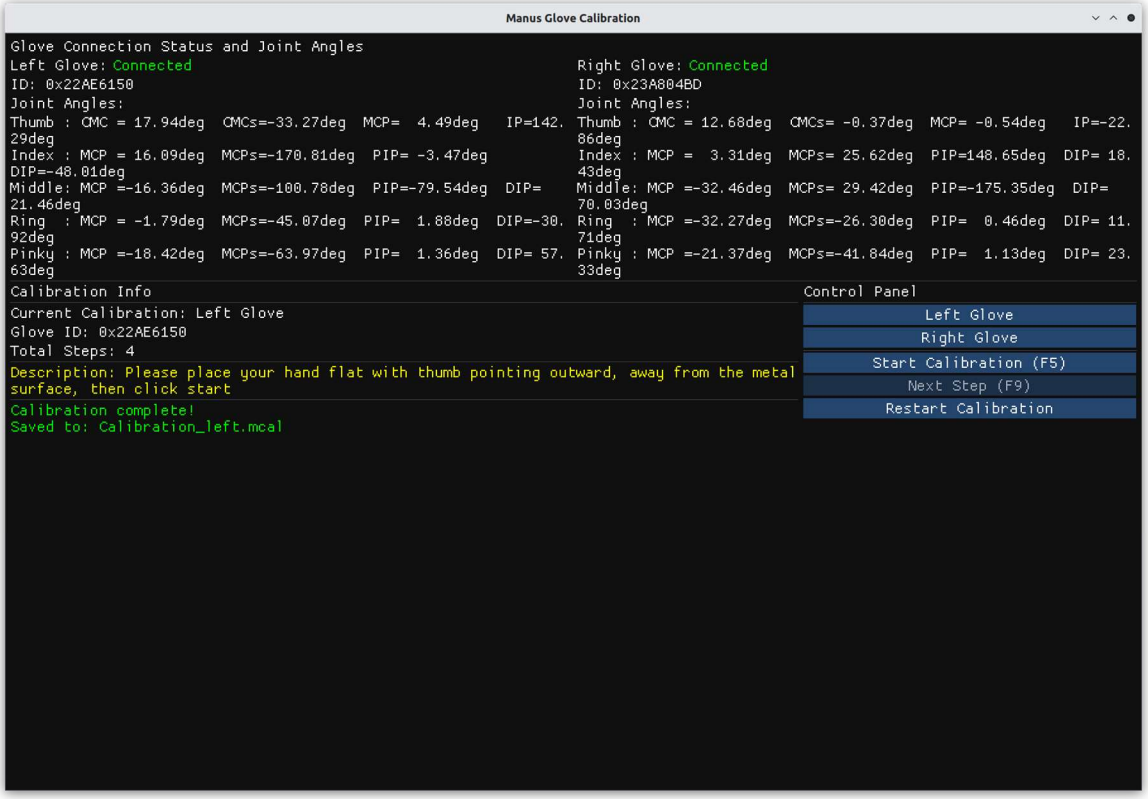


11. **Press F9 for net step as the description instructed.**

12. **Move your thumb to the base of your little finger and extend it outwards repeatedly until the description give further instruction.**



13. **When the description show “Calibration Complete”, Press F9 to finish the calibration**



14. **Click Right Glove to repeat the calibration from step 5 to step 13.**

15. **You will find the Calibration_left.mcal and Calibration_right.mcal under the current folder. Please replace the corresponding files in the SharpaManusLinuxClient folder with these two files. The new calibration file will be used after the client is restarted.**

2.5 Installation Notes

1. **If protoc is already installed on the PC and its version is lower than 3.12.4, temporarily remove it and install the latest protobuf. Detailed procedure:**

   - **Check the installed protoc version.**

```
for p in $(which -a protoc); do echo -n "$p  "; "$p" --version; done
```

   - **Example: remove protoc 3.11.2 from /usr/local/bin and the corresponding protobuf libraries.**

```
sudo mv /usr/local/bin/protoc /usr/local/bin/protoc_bak
sudo mv /usr/local/include/google /usr/local/include/google.backup
```

- **Re-compile sharpa_hand.proto under the SharpaManusLinuxClient folder and build the application.**

```
protoc --cpp_out=. sharpa_hand.proto
make clean
make all
```

- **After a successful build, restore the original 3.11.2 protobuf setup (this will not affect runtime).**

```
sudo mv /usr/local/include/google.backup /usr/local/include/google
sudo mv /usr/local/bin/protoc_bak /usr/local/bin/protoc
```

2. **Ubuntu 20.04 ships with cppzmq 4.6.0 headers, these must be manually replaced with cppzmq 4.7+ (header-only) to satisfy the API used by the retargeting module.**
   - **Download and install the header-only source archive for cppzmq 4.10.0 from the official GitHub**

```
wget https://github.com/zeromq/cppzmq/archive/v4.10.0.tar.gz
tar xf v4.10.0.tar.gz
sudo cp cppzmq-4.10.0/zmq.hpp      /usr/local/include/
sudo cp cppzmq-4.10.0/zmq_addon.hpp /usr/local/include/
```

3 Developer Guide

3 Developer Guide

3.1 Overview

Manus SDK provides the glove's raw RawSkeleton data. The Sharpa Manus Linux Client use the RawSkeleton, performs coordinate transformation on the acquired keypoints to convert them into the Sharpa standard, and then sends the resulting 25 MocapKeypoints downstream via ZMQ/Protobuf. Through a retargeting algorithm, it computes the corresponding 22 joint angles of the SharpaWave.

3.2 RawSkeleton

Refer to https://docs.manus-meta.com/latest/Plugins/SDK/SDK%20Client/#skeletons

> The second stream is the raw skeletal data, this is the glove data applied onto a standardized MANUS hand skeleton

As input to the retargeting module we need the raw skeleton—the unprocessed joint data Manus can stream immediately after calibration—not the "Retargeting Skeleton" that Manus only provides once an output skeleton model has been explicitly configured.

3.3 Data type for retargeting algs

Inputs (Mocap Keypoints)

1.Left hand keypoints: (25, 7) numpy array

```
left_keypoints = np.array([
    [x, y, z, qw, qx, qy, qz],  # keypoints 0
    [x, y, z, qw, qx, qy, qz],  # keypoints 1
    # ... 25 keypoints in total
])
```

2.Right hand keypoints: (25, 7) numpy array

```
right_keypoints = np.array([
    [x, y, z, qw, qx, qy, qz],  # keypoints 0
    [x, y, z, qw, qx, qy, qz],  # keypoints 1
```

```
    # ... 25 keypoints in total
])
```

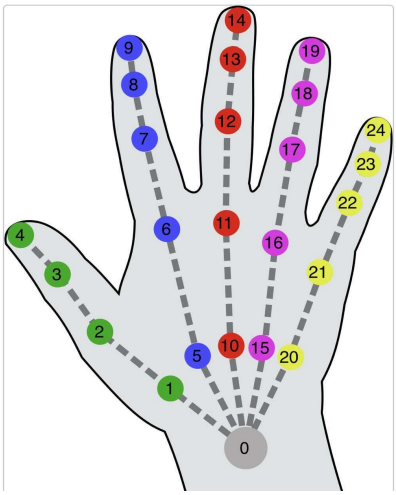
3.Keypoint definition:

- **Position: [x, y, z] (in meter)**
- **Orientation: [qw, qx, qy, qz] (quaternion)**
- **Keypoint order:**

> **1.The position and the orientation of wrist node should be set to 0 0 0 and 1 0 0 0.**
>
> **2."/" means this parameter is not used for retargeting algorithms. Only the fingertip orientations (name with _fingertip) are used for retargeting, the orientations of all other keypoints can be ignored (i.e., their quaternions may be set to 1, 0, 0, 0).**

Index	Finger	Joint_Type	Description	Position	Orientation
0	Hand	N/A	Wrist node	000	1000
1	FingerThumb	Metacarpal	First joint of the thumb	Input	/
2	FingerThumb	Proximal	Second joint of the thumb	Input	/
3	FingerThumb	Distal	Third joint of the thumb	Input	/
4	FingerThumb	Tip	Tip of the thumb	Input	Input
5	FingerIndex	Metacarpal	First joint of the index finger	Input	/
6	FingerIndex	Proximal	Second joint of the index finger	Input	/
7	FingerIndex	Intermediate	Third joint of the index finger	Input	/
8	FingerIndex	Distal	Fourth joint of the index finger	Input	/
9	FingerIndex	Tip	Tip of the index finger	Input	Input

10	FingerMiddle	Metacarpal	First joint of the middle finger	Input	/
11	FingerMiddle	Proximal	Second joint of the middle finger	Input	/
12	FingerMiddle	Intermediate	Third joint of the middle finger	Input	/
13	FingerMiddle	Distal	Fourth joint of the middle finger	Input	/
14	FingerMiddle	Tip	Tip of the middle finger	Input	Input
15	FingerRing	Metacarpal	First joint of the ring finger	Input	/
16	FingerRing	Proximal	Second joint of the ring finger	Input	/
17	FingerRing	Intermediate	Third joint of the ring finger	Input	/
18	FingerRing	Distal	Fourth joint of the ring finger	Input	/
19	FingerRing	Tip	Tip of the ring finger	Input	Input
20	FingerPinky	Metacarpal	First joint of the pinky finger	Input	/
21	FingerPinky	Proximal	Second joint of the pinky finger	Input	/
22	FingerPinky	Intermediate	Third joint of the pinky finger	Input	/
23	FingerPinky	Distal	Fourth joint of the pinky finger	Input	/
24	FingerPinky	Tip	Tip of the pinky finger	Input	Input



Keypoints order

- **Fingertip and wrist frame in Sharpa standard:**

> **Note:** All frames are right-handed. The RGB arrows denote the X-, Y-, and Z-axes, respectively. The coordinate frames of the four finger tips (except thumb) are identical, so that only the index finger and thumb are depicted in the figure.
>
> The X-axis points from the fingernail toward the pulp of the finger.

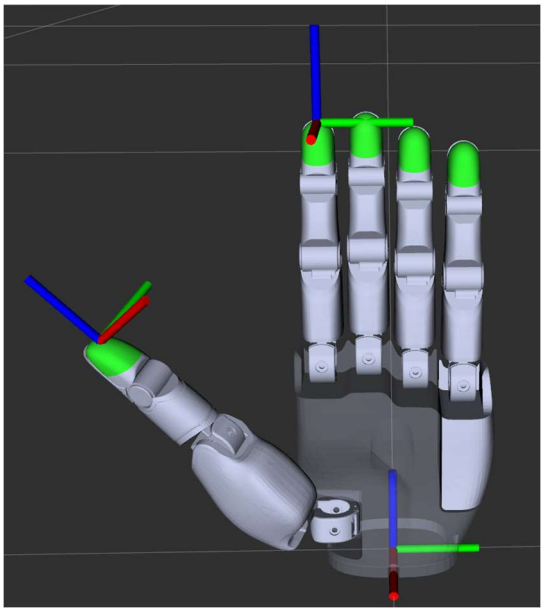


Diagram of the left-hand fingertip frame

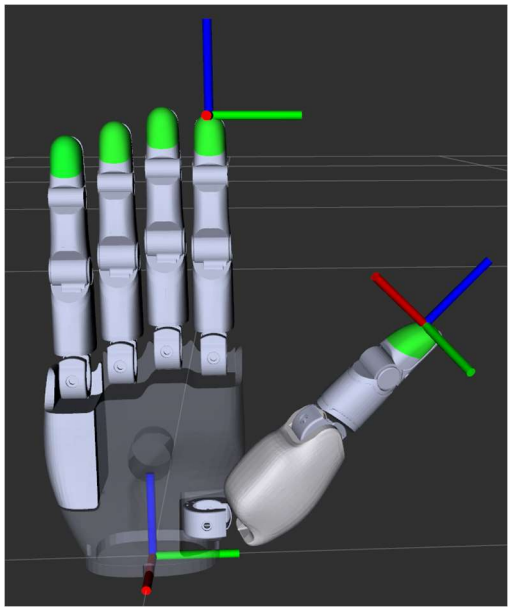


Diagram of the right-hand fingertip frame

Outputs (HA4 joints)

- Left hand joint angles `1*22 numpy array`

  `left_joint_angles = np.array([angle1, angle2, ...])`

- Right hand joint angles `1*22 numpy array`

  `right_joint_angles = np.array([angle1, angle2, ...])`

- Joint angle definition：
  - unit：`rad`
  - Joints order（right hand for example）:

index	joint_name
0	right_thumb_CMC_FE
1	right_thumb_CMC_AA
2	right_thumb_MCP_FE
3	right_thumb_MCP_AA
4	right_thumb_IP
5	right_index_MCP_FE
6	right_index_MCP_AA
7	right_index_PIP
8	right_index_DIP
9	right_middle_MCP_FE
10	right_middle_MCP_AA
11	right_middle_PIP
12	right_middle_DIP

13	right_ring_MCP_FE
14	right_ring_MCP_AA
15	right_ring_PIP
16	right_ring_DIP
17	right_pinky_CMC
18	right_pinky_MCP_FE
19	right_pinky_MCP_AA
20	right_pinky_PIP
21	right_pinky_DIP

3.4 Core functions

1.Retareget

hand_retargeting_optimizer.py

- **HandRetargetingOptimizer - single process（detail in Demo1）**

```
from hand_retargeting_optimizer import HandRetargetingOptimizer

optimizer = HandRetargetingOptimizer(hand_model, hand_type='left')

optimized_q = optimizer.optimize_with_casadi(
    initial_q=initial_angles,
    raw_keypoints=keypoints,
    keypoints_last=last_keypoints
)

cost, losses = optimizer.get_casadi_cost_value(
    optimized_q, keypoints, initial_q, last_keypoints, get_losses=True
)
```

- **MultiprocessOptimizationManager - multiprocess（detail in Demo2)**

```
from hand_retargeting_optimizer  import MultiprocessOptimizationManager

# create SharpaWave model
hand_models = init_hand_model()

# create multiprocess optimization manager
manager = MultiprocessOptimizationManager(
    hand_models,
    filter_alpha=0.2,
    hand_serial='HA4'
)

# update keypoints for optimizer manager
success = manager.update_process_keypoints('left', left_keypoints,
frame_idx)
success = manager.update_process_keypoints('right', left_keypoints,
frame_idx)

# get optimization result
result = manager.get_result('left')
result = manager.get_result('right')

# get joint angles
left_result.filtered_angles
right_result.filtered_angles
```

- **Defination**

```
joint_angles: np.ndarray      # output joint angles
filtered_angles: np.ndarray   # output joint angles after lowpass
filter
keypoints: np.ndarray         # input keypoints
cost_value: float             # total loss
optimization_time: float
frame_index: int
success: bool
losses: np.ndarray            # all losses in detail
```

3.5 Demo

Demo 1（Deprecated）: retargeting_manus_demo_simple.py - single process and no visualization

Features:

- Both-hand data are optimized within a single process, suitable for verifying that the basic retargeting function can be loaded correctly; no visualization module is included.
- HandRetargetingOptimizer is used directly.
- Optimization status and loss values are displayed in real time.

```
python3 test_zmq_sender.py

# in another terminal
python3 retargeting_manus_demo_simple.py
```

Demo 2: retargeting_manus_demo_multiprocess.py - multiprocess with visualization

Features:

- Dual-process parallel optimization for enhanced performance
- Utilizes `MultiprocessOptimizationManager`
- Visual display (requires matplotlib)

Command-Line Arguments

- **`-filter_alpha`**: Filter coefficient for smoothing (range: 0–1; default: 0.2)
  Lower values produce smoother motion. Recommended range: 0.2–0.5.
- **`-mocap_address`**: Input address for receiving motion capture (MoCap) data
  Default: `tcp://localhost:6667`
- **`-hand_action_address`**: Output address for publishing joint command messages
  Default: `tcp://*:6668`
- **`-plot`**: Enable real-time plotting and visualization
- **`-plot_orientation`**: Display orientation arrows for keypoints
  Requires `-plot` to be enabled
- **`-wave`**: Send actions directly to the physical Sharpa Wave dexterous hand via UDP using

SharpaSDK (for real-world control) When enabled, bypasses ZMQ and controls the actual hardware

- **`-glove_left_ip`: IP address assigned for the left glove, must reside in the same subnet as the Sharpa Wave IP.**

  Default: `192.168.10.99`

- **`-glove_right_ip`: IP address assigned for the right glove, must reside in the same subnet as the Sharpa Wave IP**

  Default:`192.168.10.100`

sharpa